# 《BS体系软件设计》大程设计文档

**项目名称**：智能图片管理网站
**开发者**：李禹衡
**学号**：3230105685
**专业**：计算机科学与技术
**日期**：2025年1月4日

---

## 目录

1. 项目概述
2. 系统架构设计
3. 功能模块设计
4. 数据库设计
5. API接口设计
6. 技术实现细节
7. 部署架构
8. 总结

---

## 1. 项目概述

### 1.1 项目背景

本项目是一个基于B/S架构的智能图片管理网站，旨在为用户提供便捷的图片上传、分类、检索和管理服务。系统支持PC和移动端访问，采用现代化的Web技术栈实现，具备完整的用户认证、图片处理、智能标签等核心功能。

### 1.2 项目目标

本项目完成了课程要求的所有**基本功能**：

**基本功能（11项）**

1. 用户注册、登录功能（用户名、密码至少6字节，邮箱格式验证，唯一性检查）
2. 图片上传功能（支持PC和手机浏览器）
3. EXIF信息自动提取和标签创建
4. 自定义标签功能
5. 缩略图自动生成
6. 图片信息数据库存储
7. 多条件查询界面
8. 友好的展示界面（包括轮播功能）
9. 图片编辑功能（裁剪、色调调整）
10. 删除功能
11. 移动端适配（响应式设计，支持手机浏览器和微信内置浏览器）

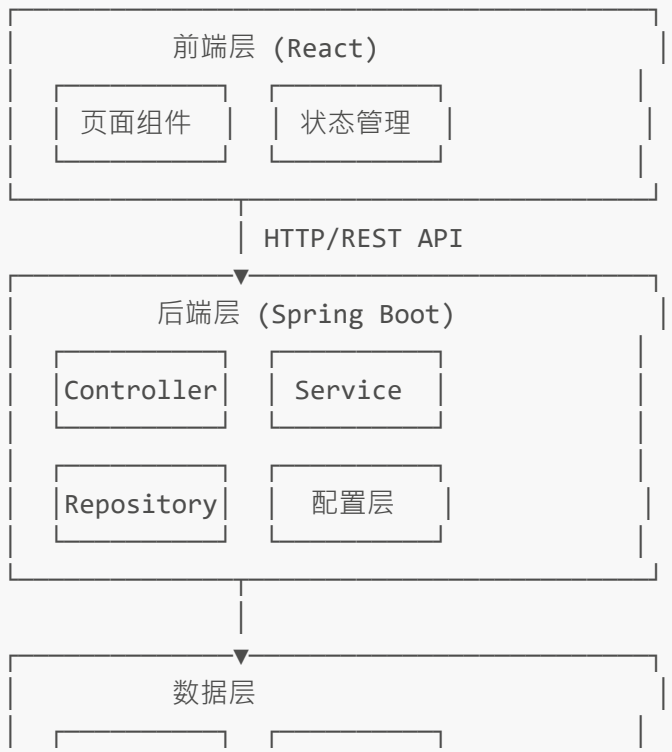
### 1.3 技术栈选型

| 层级 | 技术方案 | 说明 |
| --- | --- | --- |
| 前端框架 | React 18 | 组件化开发，生态成熟 |
| 前端UI库 | Ant Design Mobile | 移动端友好的组件库 |
| 前端构建 | Vite | 快速的构建工具 |
| 后端框架 | Spring Boot 3.2.0 | 快速开发，生态丰富 |
| 后端语言 | Java 17 | 稳定的企业级语言 |
| 数据库 | MySQL 8.0 | 稳定可靠的关系型数据库 |
| ORM框架 | Spring Data JPA | 简化数据访问 |
| 认证方式 | JWT | 无状态的认证机制 |
| 图片处理 | Thumbnailator | 缩略图生成 |
| EXIF提取 | Metadata Extractor | 图片元数据提取 |
| 容器化 | Docker + Docker Compose | 环境一致性，易于部署 |
| Web服务器 | Nginx | 前端静态文件服务 |

## 2. 系统架构设计

### 2.1 整体架构

系统采用经典的三层架构模式：



```
│  │  MySQL   │  │ 文件系统  │            │
│  └──────────┘  └──────────┘            │
└─────────────────────────────────────────┘
```

## 2.2 前端架构

### 2.2.1 技术栈

- **框架**：React 18（函数组件 + Hooks）
- **路由**：React Router v6
- **UI库**：Ant Design Mobile
- **HTTP客户端**：Axios
- **状态管理**：Context API（AuthContext）

### 2.2.2 页面结构

```
frontend/src/
├── pages/
│   ├── Login.jsx          # 登录页面
│   ├── Register.jsx       # 注册页面
│   ├── Home.jsx           # 首页
│   ├── Upload.jsx         # 图片上传页面
│   ├── Gallery.jsx        # 图片库浏览页面
│   ├── ImageDetail.jsx    # 图片详情页面
│   ├── ImageEdit.jsx      # 图片编辑页面
│   └── Slideshow.jsx      # 轮播展示页面
├── context/
│   └── AuthContext.jsx    # 认证上下文
├── utils/
│   └── api.js             # API请求封装
├── App.jsx                # 主应用组件
└── main.jsx               # 入口文件
```

### 2.2.3 关键特性

- **响应式设计**：使用CSS Media Query和Flex布局适配移动端
- **路由保护**：未登录用户自动跳转到登录页
- **Token管理**：JWT令牌存储在localStorage，自动添加到请求头
- **错误处理**：统一的错误处理和Toast提示

## 2.3 后端架构

### 2.3.1 分层架构

```
backend/src/main/java/com/zju/imagemanger/
├── config/              # 配置层
│   ├── JwtConfig.java              # JWT配置
```

```
│   ├── SecurityConfig.java         # Spring Security配置
│   ├── WebConfig.java              # Web配置（CORS等）
│   └── JwtAuthenticationFilter.java # JWT认证过滤器
├── controller/          # 控制层（REST API）
│   ├── AuthController.java         # 认证控制器
│   ├── ImageController.java        # 图片控制器
│   └── McpController.java          # MCP接口控制器
├── service/             # 服务层（业务逻辑）
│   ├── UserService.java            # 用户服务
│   ├── ImageService.java           # 图片服务
│   ├── ImageEditService.java       # 图片编辑服务
│   ├── ExifExtractor.java          # EXIF提取服务
│   ├── ThumbnailGenerator.java     # 缩略图生成服务
│   └── AiTagService.java           # AI标签服务
├── repository/          # 数据访问层
│   ├── UserRepository.java
│   ├── ImageRepository.java
│   ├── TagRepository.java
│   └── ImageTagRepository.java
├── entity/              # 实体层（数据模型）
│   ├── User.java
│   ├── Image.java
│   ├── Tag.java
│   └── ImageTag.java
├── dto/                 # 数据传输对象
│   ├── ApiResponse.java            # 统一响应格式
│   ├── AuthRequest.java            # 认证请求
│   ├── AuthResponse.java           # 认证响应
│   └── ImageEditRequest.java       # 图片编辑请求
└── exception/           # 异常处理
    └── GlobalExceptionHandler.java # 全局异常处理器
```

#### 2.3.2 关键特性

- **RESTful API设计**：统一的API路径和HTTP方法
- **统一响应格式**：所有API返回`ApiResponse<T>`格式
- **JWT认证**：使用JWT令牌进行无状态认证
- **事务管理**：使用`@Transactional`确保数据一致性
- **异常处理**：全局异常处理器统一处理错误

---

# 3. 功能模块设计

## 3.1 用户管理模块

#### 3.1.1 功能说明

处理用户注册、登录、认证等核心功能，使用JWT令牌保持登录状态。

#### 3.1.2 核心接口

**用户注册**

```
POST /api/auth/register
Content-Type: application/json

请求体：
{
  "username": "string",  // 6-50字符，字母数字下划线
  "password": "string",  // 至少6字符
  "email": "string"      // 标准邮箱格式
}

响应：
{
  "success": true,
  "message": "注册成功",
  "data": {
    "token": "jwt_token",
    "userInfo": {
      "id": 1,
      "username": "string",
      "email": "string"
    }
  }
}
```

**用户登录**

```
POST /api/auth/login
Content-Type: application/json

请求体：
{
  "username": "string",
  "password": "string"
}

响应：同注册接口
```

#### 3.1.3 验证规则

- **用户名**：6-50字符，只能包含字母、数字、下划线，必须唯一
- **密码**：至少6字符，使用BCrypt加密存储
- **邮箱**：标准邮箱格式，必须唯一

#### 3.1.4 实现细节

- 使用Spring Security进行安全配置

- JWT令牌有效期24小时
- 密码使用BCrypt进行哈希存储
- 注册时自动检查用户名和邮箱唯一性

## 3.2 图片上传模块

### 3.2.1 功能说明

支持多种图片格式上传，自动处理EXIF信息提取、缩略图生成、标签创建等。

### 3.2.2 核心接口

```
POST /api/images/upload
Content-Type: multipart/form-data
Authorization: Bearer {token}

请求参数：
- file: 图片文件（必需）
- tags: 标签数组（可选，字符串列表）

响应：
{
  "success": true,
  "message": "上传成功",
  "data": {
    "id": 1,
    "filename": "example.jpg",
    "fileSize": 1024000,
    "width": 1920,
    "height": 1080,
    "uploadTime": "2025-01-01T12:00:00"
  }
}
```

### 3.2.3 上传流程

1. **文件验证**
    - 检查文件格式（JPEG、PNG、GIF、WEBP等）
    - 检查文件大小（最大20MB）
    - 验证MIME类型
2. **文件存储**
    - 生成唯一文件名（UUID + 原文件名）
    - 按用户ID分文件夹存储
    - 保存到`uploads/{userId}/`目录
3. **图片处理**

- 提取EXIF信息（拍摄时间、相机信息、GPS等）
- 从EXIF信息自动创建标签（日期、分辨率、文件大小等）
- 调用AI服务生成AI标签
- 生成三种尺寸的缩略图（300x300、600x600、1200x1200）

4. **数据库存储**

- 保存图片基本信息到`images`表
- 保存标签到`tags`表和`image_tags`关联表
- 保存EXIF数据和AI标签到JSON字段

#### 3.2.4 实现细节

- 使用`MultipartFile`接收文件
- 使用`Thumbnailator`库生成缩略图
- 使用`Metadata Extractor`库提取EXIF信息
- 异步处理AI标签生成（提升响应速度）

### 3.3 图片浏览模块

#### 3.3.1 功能说明

提供图片列表浏览、详情查看、缩略图显示等功能。

#### 3.3.2 核心接口

**获取图片列表**

```
GET /api/images?page=0&size=20
Authorization: Bearer {token}

查询参数：
- page: 页码（默认0）
- size: 每页数量（默认20）

响应：
{
  "success": true,
  "data": {
    "content": [...],
    "totalElements": 100,
    "totalPages": 5,
    "size": 20,
    "number": 0
  }
}
```

**获取图片详情**

```
GET /api/images/{id}
Authorization: Bearer {token}

响应：
{
  "success": true,
  "data": {
    "id": 1,
    "filename": "example.jpg",
    "fileSize": 1024000,
    "width": 1920,
    "height": 1080,
    "uploadTime": "2025-01-01T12:00:00",
    "exifData": {...},
    "aiTags": {...},
    "customTags": ["标签1", "标签2"],
    "tags": ["所有标签"]
  }
}
```

**获取缩略图**

```
GET /api/images/{id}/thumbnail?size=600
Authorization: Bearer {token}

查询参数：
- size: 尺寸（300/600/1200，默认600）

响应：图片文件（JPEG格式）
```

**获取原图**

```
GET /api/images/{id}/file
Authorization: Bearer {token}

响应：原始图片文件
```

#### 3.3.3 前端实现

- **图片库页面**：网格布局展示，支持无限滚动加载
- **图片详情页面**：左右分栏布局（左侧信息，右侧图片）
- **响应式设计**：根据屏幕大小自适应布局

## 3.4 图片搜索模块

#### 3.4.1 功能说明

支持关键词搜索、时间范围搜索、标签筛选等多种搜索方式。

#### 3.4.2 核心接口

```
GET /api/images/search?keyword=xxx&startDate=xxx&endDate=xxx&page=0&size=20
Authorization: Bearer {token}

查询参数：
- keyword: 关键词（文件名模糊匹配）
- startDate: 开始时间（可选）
- endDate: 结束时间（可选）
- page: 页码
- size: 每页数量

响应：同图片列表接口
```

#### 3.4.3 前端实现

- 搜索框支持实时搜索
- 搜索结果高亮显示
- 支持按标签筛选（前端过滤）

### 3.5 图片编辑模块

#### 3.5.1 功能说明

提供图片裁剪和色调调整功能，编辑后的图片保存为新图片。

#### 3.5.2 核心接口

```
POST /api/images/{id}/edit
Content-Type: application/json
Authorization: Bearer {token}

请求体（裁剪）：
{
  "type": "CROP",
  "crop": {
    "x": 100,
    "y": 100,
    "width": 800,
    "height": 600
  },
  "tags": ["标签1"]  // 可选
}

请求体（色调调整）：
{
  "type": "ADJUST",
```

```
  "adjust": {
    "brightness": 20,   // -100 到 100
    "contrast": 10,     // -100 到 100
    "saturation": -5    // -100 到 100
  },
  "tags": ["标签1"]  // 可选
}

响应：
{
  "success": true,
  "data": {
    "id": 2,  // 新图片ID
    "filename": "example_cropped.jpg",
    ...
  }
}
```

#### 3.5.3 实现细节

**裁剪功能**

- 使用Java `BufferedImage`的`getSubimage`方法
- 验证裁剪参数的有效性
- 保存裁剪后的图片为新文件

**色调调整功能**

- 逐像素处理RGB值
- 亮度调整：RGB值乘以亮度因子
- 对比度调整：以128为基准进行线性调整
- 饱和度调整：基于灰度值进行插值

#### 3.5.4 前端实现

- 使用`react-image-crop`库实现可视化裁剪
- 使用`Slider`组件调整色调参数
- 实时预览编辑效果
- 编辑后可添加标签

### 3.6 标签管理模块

#### 3.6.1 功能说明

支持三种类型的标签：EXIF标签、AI标签、自定义标签。

#### 3.6.2 标签类型

- **EXIF标签**：从图片元数据自动提取（日期、分辨率、文件大小等）
- **AI标签**：通过AI分析自动生成（风景、人物、动物等）

- **自定义标签**：用户手动添加的标签

#### 3.6.3 核心接口

**更新图片标签**

```
PUT /api/images/{id}/tags
Content-Type: application/json
Authorization: Bearer {token}

请求体：
["标签1", "标签2", "标签3"]

响应：
{
  "success": true,
  "message": "标签更新成功"
}
```

#### 3.6.4 实现细节

- 标签存储在`tags`表，通过`image_tags`关联表与图片关联
- 标签名称不区分大小写（统一转换为小写存储）
- 删除图片时自动删除关联的标签关系
- 用户只能编辑和删除自定义标签（CUSTOM类型）
- EXIF标签和AI标签由系统自动管理，用户不能手动删除

#### 3.6.5 前端实现

- 在图片详情页面提供标签编辑功能
- 点击"编辑"按钮进入标签编辑模式
- 编辑模式下，每个自定义标签右侧显示"×"删除按钮
- 点击"×"按钮从标签列表中移除该标签
- 点击"保存"按钮将修改提交到后端
- 点击"取消"按钮放弃修改并恢复原始标签列表

### 3.7 轮播展示模块

#### 3.7.1 功能说明

支持选择多张图片进行轮播展示，自动播放。

#### 3.7.2 实现方式

**前端实现**

- 在图片库页面提供"选择"模式
- 用户可以选择多张图片

- 点击"轮播"按钮进入轮播页面
- 使用`react-image-gallery`库实现轮播

**轮播特性**

- 自动播放（3秒间隔）
- 支持缩略图导航
- 支持全屏查看
- 支持播放/暂停控制

#### 3.7.3 路由

```
/slideshow?ids=1,2,3,4
```

### 3.8 图片删除模块

#### 3.8.1 功能说明

删除图片及其所有关联数据。

#### 3.8.2 核心接口

```
DELETE /api/images/{id}
Authorization: Bearer {token}

响应：
{
  "success": true,
  "message": "删除成功"
}
```

#### 3.8.3 删除流程

1. 验证图片所有权
2. 删除文件系统中的文件（原图和所有缩略图）
3. 删除数据库记录（`images`表记录）
4. 删除标签关联（`image_tags`表记录，注意：不删除`tags`表记录，因为标签可能被其他图片使用）

## 4. 数据库设计

### 4.1 数据库选型

使用MySQL 8.0作为主数据库，存储所有结构化数据。图片文件存储在文件系统中。

### 4.2 表结构设计

#### 4.2.1 users表（用户表）

| 字段名 | 类型 | 约束 | 说明 |
|---|---|---|---|
| id | BIGINT | PRIMARY KEY, AUTO_INCREMENT | 用户ID |
| username | VARCHAR(50) | UNIQUE, NOT NULL | 用户名 |
| email | VARCHAR(100) | UNIQUE, NOT NULL | 邮箱 |
| password_hash | VARCHAR(255) | NOT NULL | 密码哈希值 |
| created_at | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP | 创建时间 |
| updated_at | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP ON UPDATE | 更新时间 |

**索引：**

- PRIMARY KEY (id)
- UNIQUE KEY (username)
- UNIQUE KEY (email)
- INDEX idx_username (username)
- INDEX idx_email (email)

#### 4.2.2 images表（图片表）

| 字段名 | 类型 | 约束 | 说明 |
|---|---|---|---|
| id | BIGINT | PRIMARY KEY, AUTO_INCREMENT | 图片ID |
| user_id | BIGINT | NOT NULL, FOREIGN KEY | 用户ID |
| filename | VARCHAR(255) | NOT NULL | 文件名 |
| file_path | VARCHAR(500) | NOT NULL | 文件路径 |
| file_size | BIGINT | NOT NULL | 文件大小（字节） |
| mime_type | VARCHAR(50) | NOT NULL | MIME类型 |
| width | INT | NULL | 图片宽度 |
| height | INT | NULL | 图片高度 |
| upload_time | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP | 上传时间 |
| exif_data | JSON | NULL | EXIF信息（JSON格式） |
| ai_tags | JSON | NULL | AI标签（JSON格式） |

**索引：**

- PRIMARY KEY (id)
- FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
- INDEX idx_user_id (user_id)
- INDEX idx_upload_time (upload_time)

#### 4.2.3 tags表（标签表）

| 字段名 | 类型 | 约束 | 说明 |
|---|---|---|---|
| id | BIGINT | PRIMARY KEY, AUTO_INCREMENT | 标签ID |
| name | VARCHAR(100) | NOT NULL | 标签名称 |
| type | ENUM('EXIF', 'AI', 'CUSTOM') | NOT NULL | 标签类型 |
| created_at | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP | 创建时间 |

**索引：**

- PRIMARY KEY (id)
- INDEX idx_name (name)
- INDEX idx_type (type)

#### 4.2.4 image_tags表（图片标签关联表）

| 字段名 | 类型 | 约束 | 说明 |
|---|---|---|---|
| image_id | BIGINT | PRIMARY KEY, FOREIGN KEY | 图片ID |
| tag_id | BIGINT | PRIMARY KEY, FOREIGN KEY | 标签ID |
| created_at | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP | 关联时间 |

**索引：**

- PRIMARY KEY (image_id, tag_id)
- FOREIGN KEY (image_id) REFERENCES images(id) ON DELETE CASCADE
- FOREIGN KEY (tag_id) REFERENCES tags(id) ON DELETE CASCADE
- INDEX idx_tag_id (tag_id)

### 4.3 关系设计

```
users (1) ──< (N) images
images (N) ──< (N) tags (通过 image_tags)
```

- 一个用户可以上传多张图片（1:N）
- 一张图片可以有多个标签，一个标签可以关联多张图片（N:M）

### 4.4 数据完整性

- 使用外键约束保证数据完整性
- 使用CASCADE删除确保关联数据一致性
- 使用唯一约束防止重复数据

---

## 5. API接口设计

## 5.1 统一响应格式

所有API接口使用统一的响应格式：

```
{
  "success": boolean,      // 请求是否成功
  "message": "string",     // 提示信息
  "data": T                // 响应数据（泛型）
}
```

## 5.2 认证接口

### 5.2.1 用户注册

```
POST /api/auth/register
Content-Type: application/json

请求体：
{
  "username": "string",
  "password": "string",
  "email": "string"
}
```

### 5.2.2 用户登录

```
POST /api/auth/login
Content-Type: application/json

请求体：
{
  "username": "string",
  "password": "string"
}
```

## 5.3 图片接口

### 5.3.1 上传图片

```
POST /api/images/upload
Content-Type: multipart/form-data
Authorization: Bearer {token}

请求参数：
```

```
- file: 图片文件
- tags: 标签数组（可选）
```

#### 5.3.2 获取图片列表

```
GET /api/images?page=0&size=20
Authorization: Bearer {token}
```

#### 5.3.3 获取图片详情

```
GET /api/images/{id}
Authorization: Bearer {token}
```

#### 5.3.4 获取缩略图

```
GET /api/images/{id}/thumbnail?size=600
Authorization: Bearer {token}
```

#### 5.3.5 获取原图

```
GET /api/images/{id}/file
Authorization: Bearer {token}
```

#### 5.3.6 搜索图片

```
GET /api/images/search?keyword=xxx&startDate=xxx&endDate=xxx&page=0&size=20
Authorization: Bearer {token}
```

#### 5.3.7 编辑图片

```
POST /api/images/{id}/edit
Content-Type: application/json
Authorization: Bearer {token}

请求体：
{
  "type": "CROP" | "ADJUST",
  "crop": {...},      // 裁剪参数
```

```
  "adjust": {...},    // 色调调整参数
  "tags": [...]       // 可选标签
}
```

#### 5.3.8 更新标签

```
PUT /api/images/{id}/tags
Content-Type: application/json
Authorization: Bearer {token}

请求体：
["标签1", "标签2", "标签3"]
```

#### 5.3.9 删除图片

```
DELETE /api/images/{id}
Authorization: Bearer {token}
```

### 5.5 错误处理

所有接口在出错时返回统一的错误响应：

```
{
  "success": false,
  "message": "错误信息",
  "data": null
}
```

常见错误码：

- 400：请求参数错误
- 401：未认证
- 403：无权限
- 404：资源不存在
- 500：服务器内部错误

---

## 6. 技术实现细节

### 6.1 安全机制

#### 6.1.1 用户认证

- 使用JWT（JSON Web Token）进行无状态认证

- Token有效期24小时
- Token存储在HTTP Header中：`Authorization: Bearer {token}`

#### 6.1.2 密码加密

- 使用BCrypt算法加密密码
- 自动生成随机盐值
- 密码永不明文存储

#### 6.1.3 权限控制

- 使用Spring Security进行权限控制
- 所有图片操作都需要验证用户所有权
- 用户只能访问自己的图片

#### 6.1.4 CORS配置

- 配置允许的前端域名
- 允许的HTTP方法：GET、POST、PUT、DELETE
- 允许的请求头：Content-Type、Authorization

### 6.2 图片处理

#### 6.2.1 缩略图生成

- 使用Thumbnailator库
- 生成三种尺寸：300x300、600x600、1200x1200
- 保持原图宽高比
- 存储格式：JPEG，质量85%

#### 6.2.2 EXIF信息提取

- 使用Metadata Extractor库
- 提取的信息：
  - 拍摄时间
  - 相机品牌和型号
  - GPS位置（如果存在）
  - 图片分辨率
  - 文件大小
- 存储格式：JSON

#### 6.2.3 图片编辑

- **裁剪**：使用Java BufferedImage API
- **色调调整**：逐像素处理RGB值
- 编辑后的图片保存为新文件
- 自动生成缩略图

### 6.3 文件存储

#### 6.3.1 存储结构

```
uploads/
  └── {userId}/
      └── {uuid}_{filename}

thumbnails/
  └── {imageId}/
      ├── thumb_300.jpg
      ├── thumb_600.jpg
      └── thumb_1200.jpg
```

#### 6.3.2 文件命名

- 原图：`{UUID}_{原文件名}`
- 编辑后图片：`{UUID}_{原文件名}_{操作类型}_{时间戳}`
- 缩略图：`thumb_{尺寸}.jpg`

### 6.4 性能优化

#### 6.4.1 前端优化

- 图片懒加载（Lazy Loading）
- 使用缩略图减少流量
- 无限滚动分页加载
- 响应式图片（根据屏幕大小选择缩略图尺寸）

#### 6.4.2 后端优化

- 数据库索引优化
- 分页查询减少数据传输
- 异步处理AI标签生成
- 缩略图缓存（文件系统缓存）

### 6.5 异常处理

#### 6.5.1 全局异常处理

- 使用`@ControllerAdvice`实现全局异常处理
- 统一错误响应格式
- 记录错误日志

#### 6.5.2 常见异常

- `IllegalArgumentException`：参数错误
- `IOException`：文件操作错误
- `AuthenticationException`：认证失败
- `AccessDeniedException`：权限不足

# 7. 部署架构

## 7.1 Docker容器化

系统使用Docker Compose进行容器化部署，包含以下服务：

1. **MySQL服务**
   - 镜像：mysql:8.0
   - 端口：3307:3306
   - 数据持久化：Docker Volume
2. **后端服务**
   - 基于Spring Boot应用
   - 端口：8080
   - 依赖：MySQL服务
3. **前端服务**
   - 基于Nginx的静态文件服务
   - 端口：3000
   - 依赖：后端服务

## 7.2 网络配置

- 所有服务在同一Docker网络中
- 前端通过Nginx反向代理访问后端API
- 后端通过服务名访问MySQL

## 7.3 数据持久化

- MySQL数据：Docker Volume持久化
- 上传文件：挂载主机目录（`./uploads`）
- 缩略图：挂载主机目录（`./thumbnails`）

## 7.4 启动方式

使用`start.bat`（Windows）或`start.sh`（Linux/Mac）一键启动所有服务。

---

# 8. 总结

## 8.1 功能完成情况

本项目完整实现了课程要求的所有**11项基本功能**：

**基本功能：**

1. 用户注册、登录（完整验证）
2. 图片上传（PC和移动端）

3. EXIF信息提取和标签创建
4. 自定义标签
5. 缩略图生成
6. 数据库存储
7. 多条件查询
8. 友好展示界面（包括轮播）
9. 图片编辑（裁剪、色调调整）
10. 删除功能
11. 移动端适配

## 8.4 后续改进方向

1. 批量上传功能
2. 图片分享功能
3. 相册管理功能
4. 更强大的图片编辑功能（滤镜、特效等）
5. 图片去重功能
6. 更智能的AI标签（接入更强大的AI模型）
7. 更完善的MCP接口（使用NLP模型进行查询解析）